Laboratorio computacional S01

Algoritmos reproducibles en R y Python

Índice general

Laboratorio computacional: R y Python **3**

Microexperimento IC10: tokenizadores comparables **5**

Protocolos complementarios de código y BPE **7**

1. Primeros pasos: tokenizar, formar ventanas y codificar **8**

1.1. El ejemplo de la guía en código sencillo 8

1.2. Lo que la representación no demuestra 8

1.3. Cómo ejecutar y comparar 8

2. Datos y convenciones compartidas **9**

2.1. R: matriz y suavizado 9

2.2. Python: matriz y suavizado 9

3. De condicionales a pérdida, PPL y BPC **10**

4. BPE con desempate definido **11**

5. Extensión a investigación: cómo empezar sin contaminar la prueba **12**

5.1. Un protocolo de documentos reales 12

5.2. Separación entre ejemplo y evidencia 12

5.3. Pruebas mínimas automatizadas 12

Laboratorio computacional: de las ecuaciones al reporte

Pregunta de trabajo

¿Una tabla de transiciones estimada con lenguaje técnico asigna mayor probabilidad a bitácoras técnicas reservadas que una tabla estimada con prosa general? Responder esta pregunta construida en pequeño ayuda a comprender qué medir, pero no demuestra nada sobre un tokenizador moderno ni sobre la población real de documentos industriales.

L1. Inspeccionar el corpus antes de ajustar el modelo

El archivo `experimentos/corpus_didactico.csv` contiene ocho documentos generales de entrenamiento, ocho técnicos de entrenamiento y seis técnicos de prueba; todas las frases fueron **redactadas para esta clase**. **No se descargaron informes de empresas ni se entrenó un LLM.** Las dos condiciones usan el mismo vocabulario de caracteres establecido *solo con entrenamiento* y el mismo valor $\alpha = 1$; los datos de prueba nunca intervienen en el ajuste ni en la selección del modelo. Se usan minúsculas y se quitan espacios de los extremos; el archivo se almacena en UTF-8 con caracteres precompuestos, sin normalización adicional.

L2. Traducir ecuaciones a un algoritmo comprobable

Para cada documento $z = z_1 \cdots z_n$, se añade un estado inicial `BOS` y se predice también un símbolo `EOS`. La probabilidad predictiva de bigramas es

$$\widehat{P}(v \mid u) = \frac{C(u, v) + \alpha}{\sum_w C(u, w) + \alpha |V_{\text{salida}}|}, \qquad \alpha = 1.$$

La pérdida en bits para el documento completo es

$$B(z) = -\sum_{t=1}^{n+1} \log_2 \widehat{P}(z_t \mid z_{t-1}), \qquad \text{BPC}(z) = \frac{B(z)}{n}.$$

Aquí $z_0 = \text{BOS}$ y $z_{n+1} = \text{EOS}$. El denominador cuenta los puntos de código del texto original (*no* EOS). **Observación:** el alfabeto es el mismo y los modelos difieren en los datos de entrenamiento, no en el algoritmo de tokenización.

L3. Ejecutar en dos lenguajes y localizar los resultados

Desde el directorio que contiene `s01_integrada.tex`:

```
python codigo/representacion.py
python codigo/s01.py
python codigo/experimento_s01.py
# En un equipo donde este instalado R:
Rscript codigo/representacion.R
Rscript codigo/s01.R
Rscript codigo/experimento_s01.R
```

Los programas nuevos exportan los resultados a `experimentos/resultados_python.csv` y `experimentos/resultados_R.csv`. Python exporta también `experimentos/resumen_python.json`. Si falta `Rscript`, la ejecución R se deja **PENDIENTE**; no se informa una comparación entre lenguajes como completada.

L4. Interpretar un resultado ejecutado, no extrapolarlo

Salida verificada por Python en este corpus sintético

Se obtuvieron seis diferencias pareadas $\Delta_i = \mathrm{BPC}_{\mathrm{tecnico}}(z_i) - \mathrm{BPC}_{\mathrm{general}}(z_i)$; su promedio fue $\bar{\Delta} \approx -0.54025$ bits por carácter. Por enumeración exhaustiva de las $6^6 = 46656$ remuestras bootstrap pareadas con reemplazo, el intervalo percentil nominal del 95 % fue $[-0.66771, -0.42038]$. El intervalo *describe el comportamiento de un procedimiento de remuestreo sobre seis frases construidas*, no la incertidumbre científica de una población de reportes: no hay muestreo representativo, comparación con modelos neuronales ni validación externa.

L5. De algoritmo a evidencia: registros obligatorios

El reporte contiene la versión del programa, datos y licencia, partición por documento, protocolo BOS/EOS, alfabeto, unidades, valores por documento, método del intervalo, limitaciones y una conclusión que no exceda lo medido. Cada cambio del corpus, semillas, modelado o tokenización requiere regenerar todas las tablas. El microexperimento IC10 evalua, por separado, la segmentacion aprendida por dos tokenizadores BPE. La consigna del taller exige un reporte corto; el seminario de investigación explica que falta para convertir una demostración en un estudio candidato a publicación.

Microexperimento IC10: cambiar realmente el tokenizador

Ahora si cambiamos la representacion, no el estimador de probabilidades

En el laboratorio anterior se compararon *dos bigramas de caracteres* con distinta procedencia de datos. Aquello ilustra estimacion de probabilidades. Aqui comparamos dos *tokenizadores BPE* que aprenden fusiones diferentes, uno en prosa general y otro en prosa tecnica, con el **mismo presupuesto de 20 fusiones**. Esta es una primera medicion de la variable de representacion que estudia IC10, sin afirmar que un modelo de lenguaje mejora por ello.

Reglas transparentes

La unidad inicial de cada palabra es un punto de código Unicode; se añade un marcador `</w>` para no fusionar entre palabras. Cada paso combina el par de unidades adyacentes mas frecuente del corpus de entrenamiento (con desempate lexicografico). Las reglas se aplican, en el orden en que se aprendieron, a las **mismas seis frases tecnicas de prueba**. La separacion entrenamiento/prueba es por documento. Se conserva la normalizacion explicita: minusculas, recorte externo de espacios y separacion por blancos. No se eliminan acentos, ni se usa el texto de prueba para elegir fusiones. Este BPE didactico no es el tokenizador de un modelo neuronal preentrenado.

Resultado verificado en Python, datos sinteticos

Tokenizador	Palabras de prueba	Tokens en prueba	Tokens/palabra
BPE entrenado con prosa general	35	148	4.228571
BPE entrenado con texto tecnico	35	127	3.628571

La diferencia agregada es $127 - 148 = -21$ tokens y $3.628571 - 4.228571 = -0.600000$ tokens por palabra, bajo estas reglas. La tabla por documento se genera en `experimentos/resultados_ic10_tokenizacion.csv`; las veinte fusiones de cada modelo, en `experimentos/fusiones_ic10.csv`.

Interpretacion responsable

El resultado no permite afirmar que un modelo general tenga peor perplejidad ni que el tokenizador especializado mejore una tarea aplicada. Para contrastar **bits por caracter** entre tokenizadores se necesita asignar probabilidades normalizadas y compatibles a las mismas secuencias (incluyendo convenciones de fronteras y fin), y evaluar en textos reservados. Menos tokens significa *otra segmentacion*; no implica por si solo menor entropia cruzada ni mejor generalizacion.

Ejecucion y reporte

```
python codigo/ic10_tokenizadores.py
# En un equipo con R instalado:
Rscript codigo/ic10_tokenizadores.R
python verificaciones/comparar_r_python.py
```

El estudiante reporta las dos comparaciones por separado: **A)** corpus de entrenamiento frente a

BPC de bigramas por caracteres; **B)** entrenamiento del tokenizador frente a tokens por palabra. Ambas son ilustrativas y no equivalen a un estudio industrial.

Protocolos complementarios de código y BPE

1 Primeros pasos: tokenizar, formar ventanas y codificar

1.1 El ejemplo de la guía en código sencillo

El siguiente ejemplo es independiente de Keras o de cualquier modelo neuronal. Solo pretende que el estudiante vea la misma operación en Python y R sobre la frase `el gato mira al perro`. La unidad es la *palabra separada por espacios*; no se presenta como un tokenizador moderno completo.

1.1.1 Python (biblioteca estándar)

```
texto = "el gato mira al perro"
tokens = texto.split()
bigramas = list(zip(tokens, tokens[1:]))
vocabulario = list(dict.fromkeys(tokens))
one_hot_gato = [int(w == "gato") for w in vocabulario]
assert len(tokens) == 5 and len(bigramas) == 4
assert sum(one_hot_gato) == 1
```

1.1.2 R (sin paquetes externos)

```
texto <- "el gato mira al perro"
tokens <- strsplit(texto, ' ', fixed=TRUE)[[1]]
bigramas <- lapply(seq_len(length(tokens)-1L),
                   function(i) tokens[i:(i+1L)])
vocabulario <- unique(tokens)
one_hot_gato <- as.integer(vocabulario == 'gato')
stopifnot(length(tokens)==5L, length(bigramas)==4L,
          sum(one_hot_gato)==1L)
```

Compare los resultados, no únicamente la apariencia del código. Los programas completos `codigo/representacion.py` y `codigo/representacion.R` muestran además trigramas, bolsa de palabras e identificadores.

1.2 Lo que la representación no demuestra

Un vector one-hot identifica un token, pero no expresa semejanza semántica ni es una probabilidad predicha. Un embedding es un vector aprendido que estudiaremos en S02. Una bolsa de palabras conserva frecuencias, no orden; y un vector por documento puede corresponder a una tarea de clasificación distinta de la predicción autorregresiva.

1.3 Cómo ejecutar y comparar

```
python codigo/representacion.py
python codigo/s01.py
python verificaciones/verificar_s01.py
Rscript codigo/representacion.R
Rscript codigo/s01.R
```

Los comandos R requieren `Rscript` en el sistema. Si no está disponible, las instrucciones se entregan con una prueba pendiente, y nunca se atribuye ejecución a R sin observarla.

2 Datos y convenciones compartidas

La cadena de entrenamiento es `abacabbacb`; el conjunto de prueba `abca`; el alfabeto $V = \{a, b, c\}$. La matriz de bigramas usa filas para el símbolo precedente y columnas para el siguiente:

$$C = \begin{pmatrix} 0 & 2 & 2 \\ 2 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix}.$$

Se adoptan logaritmos base dos para los resultados en bits y la probabilidad inicial $p(X_1 = i) = 1/3$ para la evaluación conjunta del microejemplo. Los conteos se forman solo con entrenamiento; se evalúa prueba sin alterarlos.

2.1 R: matriz y suavizado

```r
train <- strsplit("abacabbacb", "", fixed=TRUE)[[1]]
V <- c("a", "b", "c")
C <- matrix(0L, 3, 3, dimnames=list(V,V))
for (i in seq_len(length(train)-1L))
    C[train[i],train[i+1L]] <- C[train[i],train[i+1L]]+1L
P <- sweep(C+1, 1, rowSums(C)+3, "/")
stopifnot(all(abs(rowSums(P)-1)<1e-12))
```

2.2 Python: matriz y suavizado

```python
train = "abacabbacb"
V = "abc"
C = {a: {b: 0 for b in V} for a in V}
for a, b in zip(train, train[1:]):
    C[a][b] += 1
P = {a: {b: (C[a][b] + 1)/(sum(C[a].values()) + 3)
          for b in V} for a in V}
assert all(abs(sum(row.values())-1) < 1e-12
           for row in P.values())
```

Ambos programas deben obtener $P(a \to b) = 3/7$, $P(b \to c) = 1/6$ y $P(c \to a) = 2/5$. Una revisión útil consiste en contrastar no solo números redondeados, sino también suma de conteos, orden del vocabulario y convenciones sobre contextos sin observaciones.

3 De condicionales a pérdida, PPL y BPC

La probabilidad conjunta de la prueba es

$$q(\texttt{abca}) = \frac{1}{3}\frac{3}{7}\frac{1}{6}\frac{2}{5} = \frac{1}{105}.$$

```python
from math import log2
bits = log2(105)
bpc = bits / 4
ppl = 2 ** bpc
assert abs(bits - 6.714245517666122) < 1e-10
```

```r
bits <- log2(105)
bpc <- bits / 4
ppl <- 2^bpc
stopifnot(abs(bits - 6.714245517666122) < 1e-10)
```

Si se omite $\pi(a) = 1/3$ en la verosimilitud y se promedian las cuatro posiciones de la cadena, se está mezclando una pérdida *condicional* con la tasa conjunta. Es preferible reportar explícitamente la convención de evaluación.

4 BPE con desempate definido

Se procesan palabras con frecuencias *casa* (5), *casas* (2), *cosa* (6), *cosas* (3), *caso* (4). El marcador `<eow>` se incluye al final de cada palabra. El conteo de pares se pondera por frecuencia del tipo léxico; los empates se resuelven por orden Unicode ascendente sobre el par reconstruido, y los reemplazos se ejecutan de izquierda a derecha sin solapamiento.

Paso	Par	Reemplazos	Longitud
0	—	—	105
1	$a + s$	16	89
2	$a +$ fin	11	78
3	$c + as$	11	67
4	$c + o$	9	58
5	$co + s$	9	49
6	$cos + a$fin	6	43

Una fila del registro tiene dos contadores posibles: frecuencia bruta del par y número de sustituciones efectivas. Cuando existen solapamientos no tienen por qué coincidir. La comprobación correcta es

$$\text{longitud anterior} - \text{longitud nueva} = \text{número de sustituciones efectivas ponderadas}.$$

El código completo (sin omisiones) está en `codigo/s01.py` y `codigo/s01.R`. Se evitan aquí bloques extensos de código repetitivo para mantener el documento enfocado en sus invariantes y en las relaciones matemáticas.

5 Extensión a investigación: cómo empezar sin contaminar la prueba

5.1 Un protocolo de documentos reales

1. Seleccionar una fuente general y otra técnica con licencia declarada, sin suponer que el catálogo del documento de investigación contiene archivos de corpus ya descargados.
2. Congelar particiones por documento o grupo de documentos relacionados antes de aprender el vocabulario BPE.
3. Entrenar tokenización y n-gramas solo con entrenamiento; elegir suavizado y tamaños de vocabulario con validación.
4. Registrar tasa de tokens por palabra, longitud, bits por carácter y, cuando corresponda, pérdida por token compatible.
5. Presentar incertidumbre por documento o grupo, no por supuesto de independencia entre todos los tokens de un mismo documento.

5.2 Separación entre ejemplo y evidencia

La matriz, el ejemplo de KL, la tabla BPE y el contraste entre PPL son **ILUSTRATIVOS con aritmética verificada**. Una mejora en reportes técnicos se considerará **REAL** solo después de una corrida documentada sobre datos admisibles, con código y control de fuga. Ninguna de las cifras de este módulo sustituye el futuro protocolo de investigación.

5.3 Pruebas mínimas automatizadas

1. La suma de las transiciones es igual al número de símbolos menos uno.
2. Cada fila de probabilidades de una matriz válida suma uno.
3. La prueba suavizada tiene probabilidad conjunta 1/105.
4. La divergencia KL del ejemplo vale 1/4 bit.
5. El procedimiento BPE pasa de longitud 105 a 43 en seis pasos.

Estas condiciones están programadas en `verificaciones/verificar_s01.py`, que usa las mismas funciones públicas del módulo Python.